Anexo 8A

Patrones TS Avanzados en Front-end

Curso: TypeScript

180 min

Objetivos de Aprendizaje

- Implementar type guards para validación de tipos en runtime
- Crear utility types personalizados para componentes React
- Implementar contextos con tipos seguros
- Aplicar patrones de composición en React
- Implementar discriminated unions para state machines type-safe
- Crear componentes polimórficos y compound components
- Implementar validación runtime con Zod
- Crear custom hooks completamente tipados
- Implementar hook composition con tipos estrictos
- Aplicar mapped types y conditional types para utilities avanzadas
- Utilizar React 18 Concurrency (useTransition, useDeferredValue, Suspense)
- Implementar tipos en Next.js App Router
- Crear formularios tipados con React Hook Form + Zod

1. Contexto

El problema de los tipos en runtime

El Problema: Tipos en Runtime

Los datos de APIs externas no tienen tipos en tiempo de ejecución

Problema: TypeScript solo valida en compilación

```
// Datos que vienen como "any" o tipo genérico
const datos = fetch('/api/tasks'); // unknown

function procesarTarea(data: unknown) {
  // ¿Cómo sabemos qué tipo tiene data?
  // ¿Task? ¿User? ¿Algo más?
}
```

Escenario: Sistema de Gestión de Tareas

```typescript
interface Task {
  id: string;
  title: string;
  description: string;
  status: 'pending' | 'inProgress' | 'completed';
  priority: 'low' | 'medium' | 'high';
}

interface User {
  id: string;
  name: string;
  email: string;
  role: 'admin' | 'user';
}
```

Solución: Los type guards resuelven este problema

2. Type Guards

Validación de tipos en runtime

¿Qué son los Type Guards?

Funciones que validan el tipo de un valor en tiempo de ejecución

- Permiten validar el tipo en runtime
- TypeScript usa esta información para **narrowing**
- Esenciales para aplicaciones que consumen datos externos

Type Guard con typeof

```
function procesarDato(dato: string | number): string {
  if (typeof dato === 'string') {
    return dato.toUpperCase(); // TypeScript sabe que es string
  }
  return dato.toFixed(2); // TypeScript sabe que es number
}
```

Type Guard con instanceof

```
class Rectangulo {
  constructor(public ancho: number, public alto: number) {}
  area() { return this.ancho * this.alto; }
}

class Circulo {
  constructor(public radio: number) {}
  area() { return Math.PI * this.radio ** 2; }
}

type Figura = Rectangulo | Circulo;

function calcularArea(figura: Figura): number {
  if (figura instanceof Rectangulo) {
    return figura.area();
  }
  return figura.area();
}
```

Type Guard Personalizado

```
type Persona = Usuario | Administrador;

function isAdministrador(persona: Persona): persona is Administrador {
  return 'permisos' in persona && persona.tipo === 'admin';
}

function procesarPersona(persona: Persona) {
  if (isAdministrador(persona)) {
    console.log('Admin con permisos:', persona.permisos);
  } else {
    console.log('Usuario:', persona.nombre);
  }
}
```

Lab: Implementar Type Guards

```
// Type guards
function isTask(obj: unknown): obj is Task {
  return (
    typeof obj === 'object' && obj !== null &&
    'id' in obj && 'title' in obj &&
    'status' in obj && 'priority' in obj
  );
}

function isUser(obj: unknown): obj is User {
  return (
    typeof obj === 'object' && obj !== null &&
    'id' in obj && 'name' in obj && 'email' in obj
  );
}

function isTag(obj: unknown): obj is Tag {
  return (
    typeof obj === 'object' && obj !== null &&
```

3. Utility Types Personalizados

Transformando tipos para React

¿Qué son los Utility Types?

Tipos predefinidos que transforman otros tipos

- Partial<T>: hace todas las propiedades opcionales
- Pick<T, K>: selecciona solo ciertas propiedades
- Omit<T, K>: excluye ciertas propiedades

Utility Types para React

```
type ButtonProps = {
  label: string;
  onClick: () => void;
  variant: 'primary' | 'secondary';
};

type LinkButtonProps = Omit<ButtonProps, 'onClick'> & {
  href: string;
};
```

Componentes Genéricos Tipados

```
interface ListProps<T> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}

function List<T>({ items, renderItem, keyExtractor }: ListProps<T>) {
  return (
    <ul>
      {items.map(item => (
        <li key={keyExtractor(item)}>{renderItem(item)}</li>
      ))}
    </ul>
  );
}
```

Lab: Crear Utility Types

```
// types/utils.ts

// 1. Componente de lista genérico
interface ListProps<t> {
  items: T[];
  renderItem: (item: T) => React.ReactNode;
  keyExtractor: (item: T) => string;
}

// 2. Tipo para formularios
type FormState<t> = {
  values: Partial<t>;
  errors: Partial<record<keyof t,="" string="">>;
  touched: Partial<record<keyof t,="" boolean="">>;
};

// 3. Tipo para respuestas de API
type ApiResponse<t> =
  | { success: true; data: T }
```

4. Contextos Tipados

Estado global con seguridad de tipos

¿Por qué tipar el contexto?

El Context API permite compartir estado global, pero sin tipos es fácil cometer errores que solo aparecen en runtime.

- Evita errores de tipo al acceder al contexto
- Proporciona autocompletado en el IDE
- Define claramente la estructura del estado global
- El tipo del reducer define todas las acciones posibles

Contexto con Tipos

```
interface TaskManagerState {
  tasks: Task[];
  users: User[];
  tags: Tag[];
}

type TaskManagerAction =
  | { type: 'ADD_TASK'; payload: Task }
  | { type: 'REMOVE_TASK'; payload: string }
  | { type: 'SET_TAGS'; payload: Tag[] };

interface TaskManagerContextValue {
  state: TaskManagerState;
  dispatch: React.Dispatch<TaskManagerAction>;
}

const TaskManagerContext = createContext<
  TaskManagerContextValue | undefined
>(undefined);
```

Provider y Hook Personalizado

```
function TaskManagerProvider({ children }: { children: ReactNode }) {
  const [state, dispatch] = useReducer(taskManagerReducer, initialState);

  return (
    <TaskManagerContext.Provider value={{ state, dispatch }}>
      {children}
    </TaskManagerContext.Provider>
  );
}

function useTaskManager() {
  const context = useContext(TaskManagerContext);
  if (!context) {
    throw new Error('useTaskManager debe usarse dentro de TaskManagerProvider');
  }
  return context;
}
```

Lab: Implementar Contexto Tipado

```
// context/TaskContext.tsx
interface TaskState {
  tasks: Task[];
  selectedTask: Task | null;
}

type TaskAction =
  | { type: 'ADD'; payload: Omit<Task, 'id'> }
  | { type: 'UPDATE'; payload: { id: string; updates: Partial<Task> } }
  | { type: 'DELETE'; payload: string }
  | { type: 'SELECT'; payload: Task | null };

const initialState: TaskState = { tasks: [], selectedTask: null };

function taskReducer(state: TaskState, action: TaskAction): TaskState {
  switch (action.type) {
    case 'ADD':
      return { ...state, tasks: [...state.tasks, { ...action.payload, id: crypto.randomUUID() }] };
    case 'UPDATE':
```

5. Discriminated Unions

State Machines Type-Safe

¿Qué son las Discriminated Unions?

Patrón fundamental que usa una propiedad común (discriminador) para identificar la variante

- TypeScript infiere automáticamente el tipo según el discriminador
- Permite exhaustiveness checking
- Ideales para reducers y gestión de estado compleja

Estructura Básica

```
type TaskManagerAction =
  | { type: 'ADD_TASK'; payload: Task }
  | { type: 'REMOVE_TASK'; payload: string }
  | { type: 'UPDATE_TASK'; payload: Task }
  | { type: 'SET_TASKS'; payload: Task[] }
  | { type: 'ADD_TAG'; payload: Tag }
  | { type: 'SET_TAGS'; payload: Tag[] }
  | { type: 'CLEAR_ERROR' };
```

Reducer con Type Safety

```
function taskReducer(state: TaskManagerState, action: TaskManagerAction): TaskManagerState {
  switch (action.type) {
    case 'ADD_TASK':
      // TypeScript sabe que action.payload es Task
      return { ...state, tasks: [...state.tasks, action.payload] };

    case 'REMOVE_TASK':
      // TypeScript sabe que action.payload es string
      return { ...state, tasks: state.tasks.filter(t => t.id !== action.payload) };

    // ...

    default:
      // Exhaustiveness checking
      const _exhaustive: never = action;
      return state;
  }
}
```

Ventaja: Si falta un caso, error de compilación

Lab: Implementar Discriminated Unions

```
// Discriminated Union para acciones
type TaskAction =
  | { type: 'ADD'; payload: Omit<task, 'id'=""> }
  | { type: 'UPDATE'; payload: { id: string; updates: Partial<task> } }
  | { type: 'DELETE'; payload: string }
  | { type: 'SELECT'; payload: Task | null };

// Reducer con exhaustiveness checking
function taskReducer(state: TaskState, action: TaskAction): TaskState {
  switch (action.type) {
    case 'ADD':
      return { ...state, tasks: [...state.tasks, { ...action.payload, id: crypto.randomUUID() }] };
    case 'UPDATE':
      return {
        ...state,
        tasks: state.tasks.map(t => t.id === action.payload.id ? { ...t, ...action.payload.updates } : t)
      };
    case 'DELETE':
      return {    state,  tasks: state.tasks.filter(t => t.id !== action.payload) };
```

6. Componentes Avanzados

Polimórficos, Compound y forwardRef

Tipos de Componentes Avanzados

Estos patrones van más allá de componentes simples: permiten crear APIs flexibles y reutilizables que requieren tipos estrictos para funcionar correctamente.

- **Componentes polimórficos:** mismo componente puede renderizar como button, link, etc.
- **Compound components:** API declarativa donde componentes hijos se comunican con el padre
- **forwardRef:** acceder a elementos del DOM

Componente Polimórfico

```
type PolymorphicRef<C extends React.ElementType> =
  React.ComponentPropsWithRef<C>['ref'];

interface ComponentePolimorficoProps<T extends React.ElementType = 'button'> {
  as?: T;
  children: React.ReactNode;
  ref?: PolymorphicRef<T>;
}

const ComponentePolimorfico = React.forwardRef(
  <T extends React.ElementType = 'button'>(
    { as, children, ...props }: ComponentePolimorficoProps<T>,
    ref: PolymorphicRef<T>
  ) => {
    const Component = as || 'button';
    return <Component ref={ref} {...props}>{children}</Component>;
  }
);
```

Compound Components

```
function TaskManager({ children }: TaskManagerCompoundProps) {
  return <div className="task-manager">{children}</div>;
}

function TaskManagerItem({ children, value }: TaskManagerItemProps) {
  return <div data-value={value}>{children}</div>;
}

TaskManager.Item = TaskManagerItem;

// Uso
<TaskManager>
  <TaskManager.Label>My Tasks</TaskManager.Label>
  <TaskManager.Item value="task1">Learn TypeScript</TaskManager.Item>
</TaskManager>
```

forwardRef con Tipos

```
const InputBase: ForwardRefRenderFunction<InputRef, InputProps> = (
  { label, error, ...props },
  ref
) => {
  return (
    <div>
      <label>{label}</label>
      <input ref={ref} {...props} />
      {error && <span className="error">{error}</span>}
    </div>
  );
};

const Input = forwardRef(InputBase);
```

Lab: Crear Componentes Avanzados

```
// Componente polimórfico
type ButtonProps<T extends React.ElementType> = {
  as?: T;
  children: React.ReactNode;
} & Omit<React.ComponentPropsWithoutRef<T>, 'as'>;

const Button = React.forwardRef(
  <T extends React.ElementType = 'button'>(
    { as, children, ...props }: ButtonProps<T>,
    ref: React.ComponentPropsWithRef<T>['ref']
  ) => {
    const Component = as || 'button';
    return <Component ref={ref} {...props}>{children}</Component>;
  }
) as <T extends React.ElementType = 'button'>(
  props: ButtonProps<T> & { ref?: React.Ref<Element> }
) => JSX.Element;

// Compound Component
```

7. Validación Runtime con Zod

Esquemas con inferencia automática de tipos

¿Qué es Zod?

Librería de validación de esquemas que permite validar datos en runtime e inferir tipos para TypeScript

Problema resuelto: TypeScript no valida en ejecución

Esquemas con Inferencia

```
import { z } from 'zod';

const LibroSchema = z.object({
  id: z.string().uuid(),
  titulo: z.string().min(1).max(200),
  autor: z.string().min(1),
  año: z.number().int().positive(),
  disponible: z.boolean(),
});

// Inferir tipo desde schema
type Libro = z.infer<typeof LibroSchema>;

// Validar datos
function validarLibro(data: unknown): Libro {
  return LibroSchema.parse(data);
}
```

Integración con React Hook Form

```
import { zodResolver } from '@hookform/resolvers/zod';

const SocioSchema = z.object({
  nombre: z.string().min(2, 'Nombre muy corto'),
  email: z.string().email('Email inválido'),
  telefono: z.string().regex(/^\d{9}$/, 'Teléfono debe tener 9 dígitos'),
});

type SocioForm = z.infer<typeof SocioSchema>;

function FormularioSocio() {
  const { register, handleSubmit, formState: { errors } } = useForm<SocioForm>({
    resolver: zodResolver(SocioSchema),
  });
```

Lab: Implementar Validación con Zod

```
import { z } from 'zod';

// Schema para tareas
const TaskSchema = z.object({
  id: z.string().uuid().optional(),
  title: z.string().min(1, 'Título requerido').max(100),
  description: z.string().max(500).optional(),
  completed: z.boolean().default(false),
  dueDate: z.string().datetime().optional(),
  tags: z.array(z.string()).default([]),
});

type Task = z.infer<typeof TaskSchema>;

// Validación runtime
const result = TaskSchema.safeParse(taskData);
if (!result.success) {
  console.error(result.error.issues);
}
```

8. Custom Hooks Tipados

Hooks personalizados con tipos estrictos

¿Por qué tipar los hooks?

Los hooks devuelven tipos inferidos, pero cuando el tipo no es obvio (null, arrays vacíos), necesitamos ser explícitos para evitar errores en runtime.

- Proporcionan autocompletado al usar el hook
- Definen claramente qué datos devuelve el hook
- Validan que se usan correctamente
- Permiten crear hooks genéricos reutilizables

useState y useEffect con Tipos

```
// Tipado explícito
const [usuario, setUsuario] = useState<Usuario | null>(null);

// Tipado con genérico
const [items, setItems] = useState<Libro[]>([]);

function useCargarLibros() {
  const [libros, setLibros] = useState<Libro[]>([]);
  const [cargando, setCargando] = useState(true);
  const [error, setError] = useState<Error | null>(null);

  useEffect(() => {
    async function fetchLibros() {
      try {
        const response = await fetch('/api/libros');
        const data = await response.json();
        setLibros(data);
      } catch (e) {
        setError(e as Error);
```

Hooks Genéricos

```
function useColeccion<T extends { id: string }>(endpoint: string) {
  const [items, setItems] = useState<T[]>([]);
  const [cargando, setCargando] = useState(false);

  const agregar = useCallback((item: T) => {
    setItems(prev => [...prev, item]);
  }, []);

  const eliminar = useCallback((id: string) => {
    setItems(prev => prev.filter(item => item.id !== id));
  }, []);

  return { items, cargando, agregar, eliminar };
}

// Uso con tipo específico
const { items: libros, agregar, eliminar } = useColeccion<Libro>('/api/libros');
```

Lab: Crear Custom Hooks Tipados

```
// Hook genérico para colección
function useColeccion<T extends { id: string }>(endpoint: string) {
  const [items, setItems] = useState<T[]>([]);
  const [loading, setLoading] = useState(false);
  const [error, setError] = useState<Error | null>(null);

  const agregar = useCallback((item: T) =>
    setItems(prev => [...prev, item]), []);
  const eliminar = useCallback((id: string) =>
    setItems(prev => prev.filter(i => i.id !== id)), []);

  return { items, loading, error, agregar, eliminar };
}

// Hook específico para libros
function useLibros() {
  return useColeccion<Libro>('/api/libros');
}
```

9. React 18 Concurrency

Mantener la UI responsiva

¿Por qué usar Concurrencia?

React 18 introdujo capacidades que permiten trabajar en múltiples tareas simultáneamente

- **useTransition:** marca actualizaciones como "no urgentes"
- **useDeferredValue:** retrasa valores para evitar bloqueos
- **Suspense:** permite mostrar estados de carga controlados

useTransition

```
function BusquedaTareas() {
  const [busqueda, setBusqueda] = useState('');
  const [resultados, setResultados] = useState<Task[]>([]);

  const [isPending, startTransition] = useTransition();

  const handleBuscar = (query: string) => {
    setBusqueda(query);

    // La búsqueda se marca como transición
    startTransition(() => {
      const filtrados = tasks.filter(task =>
        task.title.toLowerCase().includes(query.toLowerCase())
      );
      setResultados(filtrados);
    });
  };

  return (
```

useDeferredValue

```
function ListaFiltrada({ items }: { items: Task[] }) {
  const [filtro, setFiltro] = useState('');

  // DeferredValue retrasa el valor
  const filtroDiferido = useDeferredValue(filtro);

  const itemsFiltrados = useMemo(
    () => items.filter(item =>
      item.title.includes(filtroDiferido)
    ),
    [items, filtroDiferido]
  );

  return (
    <div>
      <input value={filtro} onChange={e => setFiltro(e.target.value)} />
      <ul>{itemsFiltrados.map(item => (
        <li key={item.id}>{item.title}</li>
      ))}</ul>
```

Suspense para Data Fetching

```
function ListaTareas() {
  const tasks = use(fetchTasks());

  return (
    <ul>
      {tasks.map(task => (
        <li key={task.id}>{task.title}</li>
      ))}
    </ul>
  );
}

function PaginaTareas() {
  return (
    <Suspense fallback={<Loading />}>
      <ListaTareas />
    </Suspense>
  );
}
```

Lab: Implementar React 18 Concurrency

```
function BusquedaTareas({ tasks }: { tasks: Task[] }) {
  const [query, setQuery] = useState('');
  const [resultados, setResultados] = useState<Task[]>([]);
  const [isPending, startTransition] = useTransition();

  const debouncedQuery = useDeferredValue(query);

  useEffect(() => {
    startTransition(() => {
      setResultados(
        tasks.filter(t => t.title.toLowerCase().includes(query.toLowerCase()))
      );
    });
  }, [query, tasks]);

  return (
    <div>
      <input value={query} onChange={e => setQuery(e.target.value)} />
      {isPending ? <Spinner /> : <ul>{resultados.map(t => (
```

10. Tipos en Next.js App Router

Server y Client Components

Server vs Client Components

- **Server Components:** por defecto, pueden ser async, fetching directo a DB
- **Client Components:** necesitan directiva 'use client', usan hooks y estado
- Las props de pages y API routes tienen tipos específicos
- Server Actions requieren tipado para validación

Server Component

```
// app/tasks/page.tsx - Server Component
interface PageProps {
  params: Promise<{ estado?: string }>;
  searchParams: Promise<{ q?: string }>;
}

export default async function PaginaTareas({
  params,
  searchParams
}: PageProps) {
  const { estado } = await params;
  const { q } = await searchParams;

  const tasks = await getTasks();

  return (
    <main>
      <h1>Tareas{estado && ` - ${estado}`}</h1>
      <ListaTareas tasks={tasks} />
```

Client Component

```
// app/components/Busqueda.tsx - Client Component
'use client';

import { useState, useTransition } from 'react';

interface BusquedaProps {
  onBuscar: (query: string) => void;
}

export function Busqueda({ onBuscar }: BusquedaProps) {
  const [query, setQuery] = useState('');
  const [isPending, startTransition] = useTransition();

  return (
    <div>
      <input
        value={query}
        onChange={(e) => {
          startTransition(() => {
```

API Routes con Tipos

```
// app/api/prestamos/route.ts
const PrestamoSchema = z.object({
  idLibro: z.string().uuid(),
  idSocio: z.string().uuid(),
  dias: z.number().int().positive().max(30),
});

export async function POST(request: NextRequest) {
  const body = await request.json();
  const resultado = PrestamoSchema.safeParse(body);

  if (!resultado.success) {
    return NextResponse.json(
      { error: 'Datos inválidos', details: resultado.error.errors },
      { status: 400 }
    );
  }

  const prestamo = await crearPrestamo(resultado.data);
```

Lab: Tipos en Next.js

```
// app/libros/page.tsx - Server Component
export default async function PaginaLibros({
  searchParams
}: { searchParams: Promise<{ categoria?: string }> }) {
  const { categoria } = await searchParams;
  const libros = await getLibros(categoria);
  return (
    <main>
      <h1>Libros{categoria && ` - ${categoria}`}</h1>
      <BusquedaLibros />
      <ListaLibros libros={libros} />
    </main>
  );
}

// app/api/prestamos/route.ts
export async function POST(request: NextRequest) {
  const body = await request.json();
  const result = PrestamoSchema.safeParse(body);
```

11. Formularios Tipados

React Hook Form + Zod

¿Por qué esta combinación?

React Hook Form performance + Zod validación runtime = formularios tipados de forma automática y performant.

- **React Hook Form:** rendimiento óptimo (no re-renderiza en cada input)
- **Zod:** validación en runtime con inferencia automática
- El tipo del formulario se infiere del schema Zod
- Integración con zodResolver

Configuración Completa

```
const RegistroSocioSchema = z.object({
  nombre: z.string()
    .min(2, 'El nombre debe tener al menos 2 caracteres')
    .max(50),
  email: z.string().email('Email inválido'),
  telefono: z.string().regex(/^\d{9}$/, 'Teléfono debe tener 9 dígitos'),
  categoria: z.enum(['estudiante', 'profesor', 'externo']),
});

type RegistroSocioForm = z.infer<typeof RegistroSocioSchema>;

function FormularioRegistro() {
  const { register, handleSubmit, formState: { errors } } = useForm<RegistroSocioForm>({
    resolver: zodResolver(RegistroSocioSchema),
  });

  const onSubmit = async (data: RegistroSocioForm) => {
    await fetch('/api/socios', {
      method: 'POST'
```

Campos Dinámicos

```
function AutoresField({ control, register, errors }: AutoresFieldProps) {
  const { fields, append, remove } = useFieldArray({
    control,
    name: 'autores'
  });

  return (
    <div>
      {fields.map((field, index) => (
        <div key={field.id}>
          <input {...register(`autores.${index}.nombre` as const)} />
          <input {...register(`autores.${index}.email` as const)} />
          <button type="button" onClick={() => remove(index)}>Eliminar</button>
        </div>
      ))}
      <button type="button" onClick={() => append({ nombre: '', email: '' })}>
        Añadir autor
      </button>
    </div>
```

12. Mapped Types & Conditional Types

Transformación dinámica de tipos

¿Por qué usarlos?

Permiten crear tipos dinámicamente basándose en otros tipos: transformar, filtrar y validar en tiempo de compilación sin código runtime adicional.

- **Mapped Types:** transformar propiedades de un tipo
- **Conditional Types:** crear tipos que dependen de otros tipos
- Crear utility types personalizados
- Validar y transformar tipos en tiempo de compilación

Mapped Types

```
// Transformar todas las propiedades a opcionales
type Partial<T> = {
  [P in keyof T]?: T[P];
};

// Hacer todas las fechas strings
type FechasAString<T> = {
  [P in keyof T]: T[P] extends Date ? string : T[P];
};

interface Prestamo {
  id: string;
  fechaInicio: Date;
  fechaFin: Date;
}

type PrestamoSerializado = FechasAString<Prestamo>;
// { id: string; fechaInicio: string; fechaFin: string; }
```

Conditional Types

```
// Tipo condicional básico
type IsString<T> = T extends string ? true : false;

// Extraer el tipo de elemento de un array
type ElementType<T> = T extends (infer U)[] ? U : never;

// Obtener tipo de retorno de una función
type ReturnType<T> = T extends (...args: any[]) => infer R ? R : never;
```

Type-Safe API Responses

```typescript
type ApiSuccess<T> = {
  success: true;
  data: T;
};

type ApiError = {
  success: false;
  error: string;
  code: number;
};

type ApiResponse<T> = ApiSuccess<T> | ApiError;

function handleResponse<T>(response: ApiResponse<T>) {
  if (response.success) {
    // TypeScript sabe que response.data existe
    console.log(response.data);
  } else {
    // TypeScript sabe que response.error existe
```

Lab: Implementar Mapped Types

```typescript
// Transformar fechas a strings
type DatesToStrings<T> = { [P in keyof T]: T[P] extends Date ? string : T[P] };

// API Response type-safe
type ApiResponse<T> = { success: true; data: T } | { success: false; error: string };

// Seleccionar subconjuntos
type Subset<T, K extends keyof T> = { [P in K]: T[P] };

// Props opcionales según condición
type OptionalIfString<T, P extends keyof T> = T[P] extends string
  ? { [K in P]?: T[K] }
  : { [K in P]: T[K] };

// Uso
type LibroStrings = DatesToStrings<Libro>;
type LibroPreview = Subset<Libro, 'id' | 'titulo'>;
type PartialTitulo = OptionalIfString<Libro, 'titulo'>;
```

Resumen

Puntos Clave

- Type guards permiten validar tipos en runtime
- Utility types simplifican la manipulación de props
- Contextos tipados proporcionan seguridad en el estado global
- Componentes genéricos permiten reutilización
- Discriminated Unions: máquinas de estado type-safe
- Componentes polimórficos: flexibilidad con tipos seguros
- Compound Components: patrón de composición declarativo
- Zod: validación runtime con inferencia automática
- Custom hooks tipados: useState<T>, useEffect, useRef<T>
- Hook composition: combinar hooks para funcionalidad compleja
- Mapped Types: transformar tipos existentes
- Conditional Types: tipos condicionales

Ejercicio Integrador

Extensión del Caso 2

Objetivo

Implementar un sistema completo de biblioteca con patrones avanzados

- Sistema de validación de formularios con Zod
- Sistema de notificaciones tipado
- Lógica de negocio con type guards
- Componentes de orden superior (HOCs) tipados
- useTransition para búsquedas en tiempo real

Requisitos del Sistema

- Validar todos los datos de entrada con Zod
- Mostrar notificaciones tipadas según el tipo de operación
- Gestionar errores de forma tipada
- Utilizar useTransition para listados grandes
- Implementar componentes polimórficos
- Comportarse de forma predecible gracias a los tipos

Referencias

Recursos Adicionales

- Type Guards en TypeScript
- Utility Types
- React Context con TypeScript
- useTransition
- Zod - TypeScript-first schema validation
- Next.js App Router

Gracias!

Anexo 8A Completado

Siguiente: Anexo 12A - Testing con TypeScript